

```

1  /* =====
2      ortsveraenderung.js – Schlussakt "Ortsveränderung"
3
4      Sieben kapitelübergreifende Orte (VERGLEICHS_KNOTEN) wachsen als senkrechte
5      Linien aus der Karte, kehren auf ihre echten Koordinaten zurück, die Ansicht
6      zoomt auf sie – dann zählen die Kapitel 1..18 durch und die Kreise wachsen.
7      Ablauf in den Phasenfenstern OV_* und SK_*, umgerechnet von ovPhase().
8  ===== */
9
10 // --- Modulkapselung -----
11 // 36 von 44 Namen intern, 8 exportiert. Konvention: docs/architektur.md.
12 (function () {
13
14     // Die sieben Orte des Schlussakts, je an ihrer echten Koordinate verankert.
15     // Orte mit mehreren Namen teilen sich einen Anker (Walter: Mitte der beiden).
16     const VERGLEICHS_KNOTEN = [
17         { label: 'Redaktion La Vie Française', textSeite: 'rechts',
18           text: 'Zwölf Kapitel führen hierher. Hier wird geschrieben, was Paris für wahr hält – und hier
19             misst sich, wer er geworden ist. Der einzige Ort, der ihn weder hebt noch senkt.',
20           daten: ['12 Kapitel', '240 Annotationen', '19 neg / 19 pos', '37 F-Werte'], lon: 2.34663, lat:
21             48.87224,
22           namen: ['Redaktion La Vie Française'] },
23         { label: 'Wohnung Duroy/Madeleine (17 Rue Fontaine)', textSeite: 'rechts',
24           text: 'Er zieht in die Räume des Toten, an seinen Schreibtisch, zu seiner Frau. Was als Erbe
25             beginnt, wird zur Zelle – am Ende zählt dieser Ort nur noch Verletzungen.',
26           daten: ['9 Kapitel', '314 Annotationen', '+0.12 → -1.00', '72 F-Werte'], lon: 2.33417, lat:
27             48.88147,
28           namen: ['Wohnung Forestier (17 Rue Fontaine)', 'Wohnung Duroy/Madeleine (17 Rue Fontaine)'] },
29         { label: 'Rue Constantinople 127', textSeite: 'links-fix', versatz: { x: 0, y: 28 },
30           text: 'Die Wohnung, die niemand kennt. Erst gehört sie der Lust, dann der Berechnung – dieselben
31             vier Wände, in denen er Clotilde empfängt und Madeleine stellt.',
32           daten: ['8 Kapitel', '166 Annotationen', '43 neg / 17 pos', '43 F-Werte'], lon: 2.31921, lat:
33             48.88037,
34           namen: ['Rue Constantinople 127', 'Wohnung Du Roy (Rue Constantinople 127)'] },
35         { label: 'Palais Walter, Faubourg Saint-Honoré', textSeite: 'links-fix', versatz: { x: -28, y: 0
36           },
37           text: 'Der Gipfel, auf den er zielt, zieht selbst um – vom Boulevard ins Palais. Hier wird das
38             Vermögen gemacht, das ihn trägt, und hier bricht Frau Walter vor einem Bild zusammen, das sein
39             Gesicht hat.',
40           daten: ['6 Kapitel', '251 Annotationen', '44 neg / 23 pos', '72 F-Werte'], lon: 2.31919, lat:
41             48.87126,
42           namen: ['Boulevard Malesherbes (Walters Haus)', 'Palais Walter, Faubourg Saint-Honoré'] },
43         { label: 'Georges Duroys Wohnung (Rue Boursault)', textSeite: 'links-fix', versatz: { x: 0, y: -12
44           },
45           text: 'Ein Zimmer, das nach Armut riecht. Kein Ort des Romans wird härter empfunden – und keiner
46             verschwindet so vollständig: Nach Kapitel 7 kehrt er nie zurück.',
47           daten: ['5 Kapitel', '227 Annotationen', '86 neg / 27 pos', '98 F-Werte'], lon: 2.31879, lat:
48             48.88519,
49           namen: ['Georges Duroys Wohnung (Rue Boursault)'] },
50         { label: 'Place de la Madeleine', textSeite: 'oben-fix', versatz: { x: 28, y: 0 },
51           text: 'Am Anfang geht er hungrig an den Terrassen vorbei und zählt seine Münzen. Am Ende läutet
52             dieselbe Kirche für seine Hochzeit. Kein zweiter Ort kehrt sein Vorzeichen so vollständig um.',
53           daten: ['2 Kapitel', '106 Annotationen', '-1.00 → +0.71', '52 F-Werte'], lon: 2.32439, lat:
54             48.86993,
55           namen: ['Place de la Madeleine', 'Église de la Madeleine, Paris'] },
56         // Sammelt die kapitelweise verschiedenen Boulevard-Namen zu einer Achse.
57         // Label ist als einziges kein Kapitelname – die Gruppe hat keinen.
58         { label: 'Grands Boulevards', textSeite: 'oben-fix',
59           text: 'Die Bühne der Stadt. Erst steht er davor und sieht zu, wie andere sitzen; später sitzt er
60             selbst, bestellt und wird gesehen.',
61           daten: ['7 Kapitel', '114 Annotationen', '-0.50 → +0.50', '36 F-Werte'], lon: 2.33617, lat:
62             48.87124,
63           namen: ['Boulevard des Italiens', 'Boulevard des Capucines', 'Boulevard Poissonnière',
64             'Café Riche, Boulevard des Italiens, Paris', 'Café Tortoni, Boulevard des Italiens',
65             'Café am Boulevard (Näherung Boulevard Poissonnière)', 'Café-Chantant am Boulevard des
66             Capucines',

```

```

49     'Théâtre du Vaudeville, Boulevard des Capucines', 'Juwelierladen am Boulevard des Capucines',
50     'Boulevard-Cafés (unterwegs), Paris'] },
51 ];
52 // — Schlussakt "Ortsveränderung" —————
53
54 // Anteile am letzten Akt. Die Fenster überlappen, damit nichts hart einsetzt.
55 const OV_LINIE_WACHSEN = [0.00, 0.18];
56 const OV_KARTE_AUS     = [0.12, 0.32];
57 const OV_LINIE_ZURUECK = [0.32, 0.46];
58 const OV_LABEL_EIN     = [0.18, 0.30];
59 const OV_ZOOM          = [0.32, 0.52];
60 const OV_KAPITEL       = [0.64, 1.00];
61 // Allerletzter Akt: Startkarte blendet ein, dann Rauszoom auf die Gesamtkarte.
62 // Die Kreise verbllassen dabei, sonst liefen sie beim Rauszoomen ineinander.
63 const SK_EINBLENDEN = [0.00, 0.45];
64 const SK_RAUSZOOM   = [0.45, 1.00];
65 // Der Schlusstext kommt zuletzt und steht allein auf der Gesamtkarte.
66 const SK_TEXT       = [0.62, 0.90];
67
68 const OV_STAFFEL = 0.45; // Anteil des Wachstumsfensters, über den die Linien versetzt starten
69 const OV_ZOOM_RAND = 0.18; // Luft um die sieben Orte im Zielausschnitt
70 const OV_LINIE_BASIS = 70; // Tiefe der ersten Linie
71 const OV_LINIE_SCHRITT = 64; // zusätzliche Tiefe je weiterem Knoten
72
73 const OV_LABEL_MAX_BREITE = 200; // ab dieser Breite wird zweizeilig gesetzt
74 const OV_LABEL_ZEILE = 15; // Zeilenhöhe der Ortsbeschriftung
75 const OV_LABEL_ABSTAND = 34; // Luft zwischen Kreisrand und Ortsbeschriftung
76 const OV_LABEL_LUFT = 12; // Mindestabstand zwischen Beschriftung und nächstem Kreis
77 // Erläuterungstext je Kreis, Serifenschrift wie die Einstiegstexte, kleiner.
78 const OV_TEXT_GROESSE = 11;
79 const OV_TEXT_ZEILE = 16;
80 const OV_TEXT_BREITE = 220;
81 const OV_TEXT_ABSTAND = 30; // Luft zwischen Kreisrand und seitlichem Textblock
82 // (je Knoten über textAbstand überschreibbar)
83 const OV_TEXT_ABSTAND_OBEN = 46; // mehr Luft bei den Blöcken über dem Kreis
84 // textSeite: 'links-fix' am linken Fensterrand, 'rechts' neben dem Kreis,
85 // 'oben-fix' darüber, ohne Angabe seitlich nach aussen. Text linksbündig.
86
87 // Fixierte Varianten zählen beim Zoom nicht mit – die Kreise sollen sich
88 // ihretwegen nicht verschieben.
89 const OV_TEXT_RAND = 24; // Abstand vom Fensterrand bei fixierten Blöcken
90 // Zuführungslinie zur Textbox; die Box hängt versetzt, damit die Linie nicht
91 // durch den Text läuft.
92 const OV_LINIE_VERSATZ = 14; // Abstand zwischen Linie und erster Textzeile
93
94 // Feinkorrektur einzelner Ortspunkte in Pixeln, fährt mit dem Schlusszoom ein.
95 // Schafft Luft, wo eine Beschriftung in den Nachbarkreis lief.
96 function ovVersatz(knoten, faktor) {
97     let v = knoten.versatz;
98     return v ? { x: (v.x || 0) * faktor, y: (v.y || 0) * faktor } : { x: 0, y: 0 };
99 }
100 // Datenzeile im Stil von .annotation-tag: serifenlos, fett, versal, gesperrt.
101 const OV_DATEN_GROESSE = 9.5;
102 const OV_DATEN_ZEILE = 14;
103 const OV_DATEN_ABSTAND = 12; // Luft zwischen Fliesstext und Datenzeile
104 const OV_DATEN_TRENNER = ' . ';
105
106 // Bricht einen Fliesstext auf eine Maximalbreite um. Setzt voraus, dass
107 // Schrift und Grösse gesetzt sind (textWidth misst mit dem aktuellen Zustand).
108 function ovTextUmbruch(text, maxBreite) {
109     let zeilen = [];
110     let aktuell = '';
111     (text || '').split(' ').forEach(wort => {
112         let versuch = aktuell ? aktuell + ' ' + wort : wort;
113         if (aktuell && textWidth(versuch) > maxBreite) {
114             zeilen.push(aktuell);

```

```

115     aktuell = wort;
116   } else {
117     aktuell = versuch;
118   }
119 });
120 if (aktuell) zeilen.push(aktuell);
121 return zeilen;
122 }
123
124 // Bricht eine lange Ortsbeschriftung auf zwei Zeilen: vor Klammer, sonst nach
125 // Komma, sonst am mittigsten Leerzeichen. Schrift muss vorher gesetzt sein.
126 function ovLabelZeilen(text) {
127   if (textWidth(text) <= OV_LABEL_MAX_BREITE) return [text];
128   let klammer = text.indexOf('(');
129   if (klammer > 0) return [text.slice(0, klammer).trim(), text.slice(klammer).trim()];
130   let komma = text.indexOf(',');
131   if (komma > 0 && komma < text.length - 1) {
132     return [text.slice(0, komma + 1).trim(), text.slice(komma + 1).trim()];
133   }
134   let mitte = text.length / 2, beste = -1, abstand = Infinity;
135   for (let i = 0; i < text.length; i++) {
136     if (text[i] !== ' ') continue;
137     if (Math.abs(i - mitte) < abstand) { abstand = Math.abs(i - mitte); beste = i; }
138   }
139   if (beste < 0) return [text];
140   return [text.slice(0, beste).trim(), text.slice(beste + 1).trim()];
141 }
142
143 function ovPhase(p, fenster) {
144   return constrain(map(p, fenster[0], fenster[1], 0, 1), 0, 1);
145 }
146
147 // Je Knoten und Kapitel vorberechnet – live wären es 126 Scans pro Frame.
148 let ovProKapitel = null; // [knoten][kapitelNr] -> { bandCounts, fwerte }
149 let ovRohradian = null; // Endstand-Rohradius je Knoten (ohne Deckel)
150 let ovErstesKapitel = null; // erste Kapitelnummer mit Inhalt, je Knoten
151 let ovLayout = null; // { breite, hoehe, tiefen, kreisSkala, bbox }
152
153 function ovAddiere(ziel, quelle) {
154   ['gold_dunkel', 'gold_mittel', 'gold_hell'].forEach(cat => {
155     ['neg', 'pos', 'neutral', 'unrated'].forEach(v => { ziel[cat][v] += quelle[cat][v]; });
156   });
157 }
158
159 function ovBaueDaten() {
160   if (ovProKapitel) return;
161   ovProKapitel = VERGLEICHES_KNOTEN.map(k => {
162     let filter = new Set(k.namen);
163     let proKapitel = {};
164     let alleDaten = [['01', stationenData]].concat(
165       Object.keys(uebersichtsRouten || {}).sort().map(nr => [nr, datenFuerKapitel(nr)]));
166     alleDaten.forEach(([nr, daten]) => {
167       if (!daten || !daten.annotationen || !daten.annotationen.length) return;
168       let bis = daten.annotationen.length - 1;
169       // Ein Scan für beides. Hier unkritisch: ovBaueDaten() läuft nur einmal.
170       let treffer = sammleAnnotationenNachOrtBasis(filter, bis, daten);
171       proKapitel[nr] = {
172         bandCounts: zaehleBandCounts(treffer),
173         fwerte: treffer.filter(a => a.hasFwert),
174       };
175     });
176     return proKapitel;
177   });
178   ovRohradian = ovProKapitel.map(proKapitel => {
179     let summe = leereBandCounts();
180     Object.values(proKapitel).forEach(k => ovAddiere(summe, k.bandCounts));

```

```

181 // Infinity statt des 100px-Deckels – sonst wären am Ende fast alle Orte
182 // gleich gross. Verkleinert wird gemeinsam über kreisSkala.
183 return groessterKreisRadius(summe, Infinity);
184 });
185 // Erstes Kapitel, in dem der Ort überhaupt vorkommt – daran hängt das
186 // Einblenden seiner Textbox (siehe zeichneOrtsveraenderung).
187 ovErstesKapitel = ovProKapitel.map(proKapitel => {
188   let erstes = 18;
189   Object.keys(proKapitel).sort().forEach(nr => {
190     if (groessterKreisRadius(proKapitel[nr].bandCounts, Infinity) > 0) erstes = Math.min(erstes,
191       parseInt(nr, 10));
192   });
193   return erstes;
194 });
195 ovLayout = null;
196 }
197
198 // Summe bis einschliesslich Kapitel maxNr, plus die Nummer des letzten
199 // Kapitels, das beigetragen hat (steht unter dem Kreis).
200 function ovStand(index, maxNr) {
201   let summe = leereBandCounts();
202   let fwerte = [];
203   let letztes = null;
204   Object.keys(ovProKapitel[index]).sort().forEach(nr => {
205     if (parseInt(nr, 10) > maxNr) return;
206     let k = ovProKapitel[index][nr];
207     ovAddiere(summe, k.bandCounts);
208     fwerte = fwerte.concat(k.fwerte);
209     let n = (summe.gold_dunkel.neg + summe.gold_dunkel.pos + summe.gold_dunkel.neutral +
210       summe.gold_dunkel.unrated)
211       + (summe.gold_mittel.neg + summe.gold_mittel.pos + summe.gold_mittel.neutral +
212       summe.gold_mittel.unrated)
213       + (summe.gold_hell.neg + summe.gold_hell.pos + summe.gold_hell.neutral +
214       summe.gold_hell.unrated);
215     if (n > 0 && k.fwerte.length + 1 > 0) letztes = nr;
216   });
217   return { bandCounts: summe, fwerte, letztes };
218 }
219
220 // Zielausschnitt des Schlusszooms: alle sieben Orte plus Rand, aufs
221 // Canvas-Seitenverhältnis gebracht, dazu die grösstmögliche Kreis-Skala.
222 function ovBerechneLayout() {
223   if (ovLayout && ovLayout.breite === width && ovLayout.hoehe === height) return ovLayout;
224
225   const mercY = (lat) => Math.log(Math.tan(Math.PI / 4 + radians(lat) / 2));
226   const mercLat = (y) => degrees(2 * Math.atan(Math.exp(y)) - Math.PI / 2);
227   const RAND = 12; // Mindestluft zum Fensterrand
228
229   // Beschriftungsbreiten messen: sie können breiter sein als ihr Kreis und
230   // bestimmen mit, wie weit ein Ort vom Rand weg muss.
231   textFont(SHRIFT_SANS);
232   textStyle(BOLD);
233   textSize(13);
234   let labelZeilen = VERGLEICHES_KNOTEN.map(k => ovLabelZeilen(k.label.toUpperCase()));
235   let labelHalb = labelZeilen.map(z => Math.max(...z.map(t => textWidth(t))) / 2);
236   textStyle(NORMAL);
237
238   // Erläuterungstexte umbrechen – in ihrer eigenen Schrift gemessen.
239   textFont(SHRIFT_SERIF);
240   textSize(OV_TEXT_GROESSE);
241   let textZeilen = VERGLEICHES_KNOTEN.map(k => ovTextUmbruch(k.text, OV_TEXT_BREITE));
242   textFont(SHRIFT_SANS);
243   textStyle(BOLD);
244   textSize(OV_DATEN_GROESSE);
245   // Zwei feste Zeilen: Kapitel/Annotationen oben, Valenz/F-Werte darunter –
246   // so steht die Gefühlsangabe bei jedem Ort an derselben Stelle.

```

```

243   let datenZeilen = VERGLEICHES_KNOTEN.map(k => {
244     if (!k.daten || !k.daten.length) return [];
245     let oben = k.daten.slice(0, 2).join(OV_DATEN_TRENNER).toUpperCase();
246     let unten = k.daten.slice(2).join(OV_DATEN_TRENNER).toUpperCase();
247     return ovTextUmbruch(oben, OV_TEXT_BREITE).concat(unten ? ovTextUmbruch(unten, OV_TEXT_BREITE) :
[]);
248   });
249   textStyle(NORMAL);
250   textFont(SCHRIFT_SERIF);
251   textSize(OV_TEXT_GROESSE);
252
253   // Höhe unter dem Kreis: nur Ortsbeschriftung und Kapitelzeile.
254   let untenHoehe = labelZeilen.map(z => OV_LABEL_ABSTAND + (z.length - 1) * OV_LABEL_ZEILE + 16 +
7);
255   let textBreite = textZeilen.map(z => z.length ? Math.max(...z.map(t => textWidth(t))) : 0);
256   // Halbe Texthöhe – der Block steht mittig auf der Kreishöhe.
257   let textHalbHoehe = textZeilen.map(z => z.length ? (z.length - 1) * OV_TEXT_ZEILE / 2 : 0);
258
259   let lons = VERGLEICHES_KNOTEN.map(k => k.lon);
260   let lats = VERGLEICHES_KNOTEN.map(k => k.lat);
261   let w0 = Math.min(...lons), o0 = Math.max(...lons);
262   let s0 = Math.min(...lats), n0 = Math.max(...lats);
263
264   // Bbox für einen gegebenen Rand bauen: Rand auf die Ankerspanne, dann auf
265   // das Canvas-Seitenverhältnis aufziehen.
266   //
267   // ACHTUNG Seitenverhältnis in MERCATOR-Einheiten, nicht in Grad – sonst
268   // wird der Ausschnitt auf Pariser Höhe um Faktor 1.52 horizontal gestreckt.
269   let baueBbox = (rand) => {
270     let dLon = (o0 - w0) * rand, dLat = (n0 - s0) * rand;
271     let west = w0 - dLon, ost = o0 + dLon, sued = s0 - dLat, nord = n0 + dLat;
272     let xW = radians(west), x0 = radians(ost);
273     let yS = mercY(sued), yN = mercY(nord);
274     let sv = width / height;
275     if ((x0 - xW) / (yN - yS) < sv) {
276       let zu = ((yN - yS) * sv - (x0 - xW)) / 2;
277       west = degrees(xW - zu); ost = degrees(x0 + zu);
278     } else {
279       let zu = ((x0 - xW) / sv - (yN - yS)) / 2;
280       sued = mercLat(yS - zu); nord = mercLat(yN + zu);
281     }
282     return { west, east: ost, south: sued, north: nord };
283   };
284
285   // Grösster Faktor, bei dem sich weder Kreise berühren noch eine
286   // Beschriftung in den Kreis darunter läuft.
287
288   // ACHTUNG zweite Bedingung nötig, weil der Text unter dem Kreis NICHT
289   // mitskaliert: dy >= r_oben*s + untenHoehe_oben + r_unten*s, also
290   // s <= (dy - untenHoehe_oben) / (r_oben + r_unten).
291   let skalaFuer = (bbox, pos) => {
292     let skala = 1;
293     for (let i = 0; i < pos.length; i++) {
294       for (let j = i + 1; j < pos.length; j++) {
295         let d = dist(pos[i].x, pos[i].y, pos[j].x, pos[j].y);
296         skala = Math.min(skala, d / ((ovRohradien[i] + ovRohradien[j]) * 1.06));
297       }
298       let oben = pos[i].y < pos[j].y ? i : j;
299       let unten = oben === i ? j : i;
300       let dy = Math.abs(pos[i].y - pos[j].y);
301       let dx = Math.abs(pos[i].x - pos[j].x);
302       // Trifft die Beschriftung des oberen den unteren überhaupt seitlich?
303       if (dx < labelHalb[oben] + ovRohradien[unten] * skala) {
304         // OV_LABEL_LUFT: sonst berühren sich Beschriftung und Kreis exakt,
305         // weil die Bedingung mit Gleichheit erfüllt wird.

```

```

306     let erlaubt = (dy - untenHoehe[oben] - OV_LABEL_LUFT) / (ovRohradien[oben] +
ovRohradien[unten]);
307     if (erlaubt > 0) skala = Math.min(skala, erlaubt);
308   }
309 }
310 }
311 return skala;
312 };
313
314 // Überstand über den Fensterrand in px: Kreisradius nach allen Seiten, unten
315 // zusätzlich Beschriftung und Kapitelzeile, seitlich die halbe Labelbreite.
316
317 // Erläuterungstext nach AUSSEN – er wandert so nie über die Bildmitte, wo
318 // die meisten Nachbarkreise liegen.
319 let textRechts = pos => pos.x < width / 2;
320 let ueberstand = (pos, skala) => {
321   let max = 0;
322   pos.forEach((p, i) => {
323     let r = ovRohradien[i] * skala;
324     let eigen = Math.max(r, labelHalb[i]);
325     // Am Fensterrand fixierte Blöcke bleiben aussen vor: sie stehen fest und
326     // dürfen den Zoom nicht aufziehen.
327     let fix = !!VERGLEICH_KNOTEN[i].textSeite;
328     let block = fix ? eigen : r + OV_TEXT_ABSTAND + textBreite[i];
329     let links = textRechts(p) ? eigen : block;
330     let rechts = textRechts(p) ? block : eigen;
331     max = Math.max(max,
332       RAND - (p.y - Math.max(r, textHalbHoehe[i])), // oben
333       (p.y + r + untenHoehe[i]) - (height - RAND), // unten
334       (p.y + textHalbHoehe[i]) - (height - RAND), // Textblock nach unten
335       RAND - (p.x - links),
336       (p.x + rechts) - (width - RAND));
337   });
338   return max;
339 };
340
341 // Rand aufziehen, bis alles passt. Grösserer Rand heisst kleinere Kreise,
342 // deshalb konvergiert das.
343 let rand = OV_ZOOM_RAND;
344 let mitVersatz = (bbox) => VERGLEICH_KNOTEN.map(k => {
345   let p = lonLatToScreen(k.lon, k.lat, bbox, 0, 0);
346   let v = ovVersatz(k, 1); // das Layout rechnet mit dem Endzustand
347   return { x: p.x + v.x, y: p.y + v.y };
348 });
349 let bbox = baueBbox(rand);
350 let pos = mitVersatz(bbox);
351 let kreisSkala = skalaFuer(bbox, pos);
352 for (let versuch = 0; versuch < 20 && ueberstand(pos, kreisSkala) > 0.5; versuch++) {
353   rand *= 1.15;
354   bbox = baueBbox(rand);
355   pos = mitVersatz(bbox);
356   kreisSkala = skalaFuer(bbox, pos);
357 }
358
359 // Tiefe der senkrechten Linien in der Zwischenphase: gestaffelt nach der
360 // Höhenlage der Anker, gedeckelt auf die Fensterhöhe.
361 let reihenfolge = pos.map((p, i) => ({ i, y: p.y })).sort((a, b) => a.y - b.y).map(a => a.i);
362 let tiefen = [];
363 reihenfolge.forEach((idx, rang) => { tiefen[idx] = OV_LINIE_BASIS + rang * OV_LINIE_SCHRITT; });
364 let maxTiefe = Math.max(...tiefen);
365 let platz = height - 90;
366 if (maxTiefe > platz) tiefen = tiefen.map(t => t * platz / maxTiefe);
367
368 ovLayout = { breite: width, hoehe: height, bbox, kreisSkala, tiefen, reihenfolge, rand,
369   labelZeilen, textZeilen, datenZeilen };
370 return ovLayout;

```

```

371 }
372
373 // Zielausschnitt für den Schlusszoom – draw() blendet activeBbox dorthin.
374 function ovZoomBbox() {
375     if (!ovProKapitel) return null;
376     return ovBerechneLayout().bbox;
377 }
378
379 // p = Fortschritt im Akt (0..1), bbox = die gerade gültige Kartenbbox.
380 // textFaktor blendet NUR die Schrift aus, die Kreise bleiben stehen.
381 function zeichneOrtsveraenderung(bbox, p, alpha, textFaktor = 1) {
382     if (alpha <= 0 || !stationenData || !stationenData.annotationen) return;
383     ovBaueDaten();
384     let layout = ovBerechneLayout();
385
386     let pZoomPhase = ovPhase(p, OV_ZOOM);
387     let pWachsen = ovPhase(p, OV_LINIE_WACHSEN);
388     let pZurueck = ovPhase(p, OV_LINIE_ZURUECK);
389     let pLabel = ovPhase(p, OV_LABEL_EIN);
390     let pKapitel = ovPhase(p, OV_KAPITEL);
391     let maxKapitel = Math.max(1, Math.min(18, Math.ceil(pKapitel * 18)));
392     let kreisAlpha = pKapitel > 0 ? 1 : 0;
393
394     let n = VERGLEICHES_KNOTEN.length;
395
396     textFont(SCHRIFT_SANS);
397     textAlign(CENTER, CENTER);
398
399     VERGLEICHES_KNOTEN.forEach((k, i) => {
400         let anker = lonLatToScreen(k.lon, k.lat, bbox, 0, 0);
401         let v = ovVersatz(k, pZoomPhase);
402         anker = { x: anker.x + v.x, y: anker.y + v.y };
403         // Gestaffelter Start: der Knoten mit dem obersten Anker beginnt zuerst.
404         let rang = layout.reihenfolge.indexOf(i);
405         let start = (rang / n) * OV_STAFFEL;
406         let wachsen = constrain(map(pWachsen, start, start + (1 - OV_STAFFEL), 0, 1), 0, 1);
407         let tiefe = layout.tiefen[i] * wachsen * (1 - pZurueck);
408         let cy = anker.y + tiefe;
409
410         if (tiefe > 0.5) {
411             stroke(ROUTE_COLOR_RGB.r, ROUTE_COLOR_RGB.g, ROUTE_COLOR_RGB.b, alpha);
412             strokeWeight(1.5);
413             line(anker.x, anker.y, anker.x, cy);
414             noStroke();
415         }
416
417         // Ortspunkt am unteren Ende der Linie – er wandert hinunter und wieder
418         // zurück auf seinen echten Ort.
419         drawingContext.fillStyle = `rgba(${ROUTE_COLOR_RGB.r}, ${ROUTE_COLOR_RGB.g},
420     ${ROUTE_COLOR_RGB.b}, ${alpha / 255})`;
421         drawingContext.beginPath();
422         drawingContext.arc(anker.x, cy, 3.5, 0, TWO_PI);
423         drawingContext.fill();
424
425         let radius = 0;
426         let stand = null;
427         if (kreisAlpha > 0) {
428             stand = ovStand(i, maxKapitel);
429             // Innerhalb des Blocks: bei unsichtbarem Kreis muss radius 0 bleiben,
430             // daran hängt der Abstand der Beschriftung darunter.
431             radius = groessterKreisRadius(stand.bandCounts, Infinity, layout.kreisSkala);
432             zeichneKreiseFuerRun(anker.x, cy, stand.bandCounts, (alpha / 255) * kreisAlpha,
433                 PI, layout.kreisSkala, Infinity);
434             zeichneWertPunkte(anker.x, cy, radius, stand.fwerte, (alpha / 255) * kreisAlpha);
435         }
436     });

```



```

436 // Reihenfolge von oben nach unten: Kreis, Ortsbeschriftung, Kapitelzähler.
437 let rand = radius > 0 ? radius : 6;
438
439 let zeilen = layout.labelZeilen[i];
440 if (pLabel > 0) {
441     textStyle(BOLD);
442     textSize(13);
443     // ACHTUNG fillStyle direkt setzen, nicht über fill(): p5 überspringt die
444     // Zuweisung bei gleichbleibendem Wert. zeichneFwertPunkte schreibt oben
445     // direkt in fillStyle und umgeht den Zwischenspeicher – der Ortsname
446     // wurde dadurch rot, sobald sich seine Deckkraft nicht mehr änderte.
447     drawingContext.fillStyle = `rgba(33, 43, 46, ${alpha * pLabel * textFaktor / 255})`;
448     zeilen.forEach((zeile, z) => {
449         drawingContext.fillText(zeile, anker.x, cy + rand + OV_LABEL_ABSTAND + z * OV_LABEL_ZEILE);
450     });
451 }
452
453 // Kapitelzähler darunter – rutscht bei zweizeiliger Beschriftung mit.
454 if (kreisAlpha > 0 && stand) {
455     textStyle(NORMAL);
456     textSize(11);
457     drawingContext.fillStyle = `rgba(90, 90, 90, ${alpha * kreisAlpha * textFaktor / 255})`;
458     drawingContext.fillText(stand.letztes ? `Kapitel ${stand.letztes}` : 'Kapitel –',
459         anker.x, cy + rand + OV_LABEL_ABSTAND + (zeilen.length - 1) * OV_LABEL_ZEILE + 16);
460 }
461
462 // Erläuterung seitlich neben dem Kreis, linksbündig und nach aussen.
463
464 // Die Textbox erscheint, wenn dieser Ort seine erste Annotation bekommt –
465 // die Texte kommen dadurch nacheinander statt alle auf einmal.
466 let erstes = ovErstesKapitel[i];
467 let pTextbox = constrain(map(pKapitel * 18, erstes - 1, erstes, 0, 1), 0, 1);
468 let erlaeuterung = layout.textZeilen[i];
469 if (pTextbox > 0 && erlaeuterung && erlaeuterung.length) {
470     textFont(SCHRIFT_SERIF);
471     textStyle(NORMAL);
472     textSize(OV_TEXT_GROESSE);
473     textAlign(LEFT, CENTER);
474     drawingContext.textAlign = 'left';
475     drawingContext.fillStyle = `rgba(33, 43, 46, ${alpha * pTextbox * textFaktor * 0.85 / 255})`;
476     let breite = Math.max(...erlaeuterung.map(t => textWidth(t)));
477     // Am ENDRADIUS aufgehängt, nicht am laufenden – sonst schöbe der
478     // wachsende Kreis den Text Kapitel für Kapitel vor sich her.
479     let endRand = ovRohr radien[i] * layout.kreisSkala;
480     let abstand = k.textAbstand || OV_TEXT_ABSTAND;
481     let linksBuendig, start;
482     let linie = null; // [x1, y1, x2, y2] – Zuführung vom Ortspunkt zur Box
483     if (k.textSeite === 'oben-fix') {
484         // Senkrechte Linie vom Kreis nach oben; die Box hängt rechts davon,
485         // ihr oberer Rand am Ende der Linie.
486         let datenHoehe = layout.datenZeilen[i].length
487             ? OV_DATEN_ABSTAND + layout.datenZeilen[i].length * OV_DATEN_ZEILE : 0;
488         start = cy - endRand - OV_TEXT_ABSTAND_OBEN - datenHoehe
489             - (erlaeuterung.length - 1) * OV_TEXT_ZEILE;
490         linksBuendig = anker.x + OV_LINIE_VERSATZ;
491         linie = [anker.x, cy - endRand, anker.x, start - OV_TEXT_ZEILE / 2];
492     } else {
493         // Waagrechte Linie auf Punkthöhe, Box darunter. Sie spannt über die
494         // Boxbreite, damit der Text frei bleibt.
495         let rechtsDavon = k.textSeite === 'rechts' ? true
496             : anker.x < width / 2;
497         if (k.textSeite === 'links-fix') linksBuendig = OV_TEXT_RAND;
498         else linksBuendig = rechtsDavon
499             ? anker.x + endRand + abstand
500             : anker.x - endRand - abstand - breite;
501         let nachRechts = linksBuendig > anker.x;

```



```

502     linie = [anker.x + (nachRechts ? endRand : -endRand), cy,
503             nachRechts ? linksBuendig + breite : linksBuendig, cy];
504     start = cy + OV_LINIE_VERSATZ + OV_TEXT_ZEILE / 2;
505 }
506
507 if (linie) {
508     stroke(33, 43, 46, alpha * pTextbox * textFaktor * 0.5);
509     strokeWeight(1);
510     line(linie[0], linie[1], linie[2], linie[3]);
511     noStroke();
512 }
513 erlaeuterung.forEach((zeile, z) => {
514     drawingContext.fillText(zeile, linksBuendig, start + z * OV_TEXT_ZEILE);
515 });
516
517 // Datenzeile darunter. letterSpacing kennt nicht jeder Browser, wo es
518 // fehlt wird es ignoriert.
519 let daten = layout.datenZeilen[i];
520 if (daten && daten.length) {
521     textFont(SCHRIFT_SANS);
522     textStyle(BOLD);
523     textSize(OV_DATEN_GROESSE);
524     drawingContext.letterSpacing = '0.06em';
525     drawingContext.fillStyle = `rgba(33, 43, 46, ${alpha * pTextbox * textFaktor * 0.7 / 255})`;
526     let dStart = start + (erlaeuterung.length - 1) * OV_TEXT_ZEILE + OV_DATEN_ABSTAND +
OV_DATEN_ZEILE;
527     daten.forEach((zeile, z) => {
528         drawingContext.fillText(zeile, linksBuendig, dStart + z * OV_DATEN_ZEILE);
529     });
530     drawingContext.letterSpacing = '0px';
531     textStyle(NORMAL);
532 }
533
534 textFont(SCHRIFT_SANS);
535 textSize(11);
536 textAlign(CENTER, CENTER);
537 drawingContext.textAlign = 'center';
538 }
539 });
540
541 textAlign(LEFT, CENTER); // zurücksetzen – andere Zeichenfunktionen erwarten das
542 textStyle(NORMAL);
543 }
544
545
546 // --- Export -----
547 // Acht Namen, alle nur von sketch.js gelesen.
548 window.OV_KARTE_AUS = OV_KARTE_AUS;
549 window.OV_ZOOM = OV_ZOOM;
550 window.SK_EINBLENDEN = SK_EINBLENDEN;
551 window.SK_RAUSZOOM = SK_RAUSZOOM;
552 window.SK_TEXT = SK_TEXT;
553 window.ovPhase = ovPhase;
554 window.ovZoomBbox = ovZoomBbox;
555 window.zeichneOrtsveraenderung = zeichneOrtsveraenderung;
556
557 }()); // Ende der Modulkapselung, siehe Kommentar oben

```